

ML3_TITANIC

October 19, 2025

```
[84]: !pip install lime
      !pip install shap
      !pip install sklearn
      ### IMPORTAR LIBRERIAS ###
      import pandas as pd
      import numpy as np
      import seaborn as sns
      import matplotlib.pyplot as plt

      from sklearn import tree
      from sklearn.model_selection import train_test_split
      from sklearn.pipeline import Pipeline, make_pipeline
      from sklearn.impute import SimpleImputer
      from sklearn.preprocessing import StandardScaler
      from sklearn.linear_model import LogisticRegression
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.metrics import (classification_report, accuracy_score,
      ↪confusion_matrix,
      RocCurveDisplay, PrecisionRecallDisplay)
      from sklearn import tree
```

Requirement already satisfied: lime in c:\users\surface\anaconda3\lib\site-packages (0.2.0.1)
Requirement already satisfied: matplotlib in c:\users\surface\anaconda3\lib\site-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in c:\users\surface\anaconda3\lib\site-packages (from lime) (2.1.3)
Requirement already satisfied: scipy in c:\users\surface\anaconda3\lib\site-packages (from lime) (1.15.3)
Requirement already satisfied: tqdm in c:\users\surface\anaconda3\lib\site-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in c:\users\surface\anaconda3\lib\site-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in c:\users\surface\anaconda3\lib\site-packages (from lime) (0.25.0)
Requirement already satisfied: networkx>=3.0 in c:\users\surface\anaconda3\lib\site-packages (from scikit-image>=0.12->lime) (3.4.2)

Requirement already satisfied: pillow>=10.1 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-image>=0.12->lime)
(11.1.0)

Requirement already satisfied: imageio!=2.35.0,>=2.33 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-image>=0.12->lime)
(2.37.0)

Requirement already satisfied: tifffile>=2022.8.12 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-image>=0.12->lime)
(2025.2.18)

Requirement already satisfied: packaging>=21 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-image>=0.12->lime)
(24.2)

Requirement already satisfied: lazy-loader>=0.4 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-image>=0.12->lime)
(0.4)

Requirement already satisfied: joblib>=1.2.0 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-learn>=0.18->lime)
(1.4.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-learn>=0.18->lime)
(3.5.0)

Requirement already satisfied: contourpy>=1.0.1 in
c:\users\surface\anaconda3\lib\site-packages (from matplotlib->lime) (1.3.1)

Requirement already satisfied: cyclor>=0.10 in
c:\users\surface\anaconda3\lib\site-packages (from matplotlib->lime) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in
c:\users\surface\anaconda3\lib\site-packages (from matplotlib->lime) (4.55.3)

Requirement already satisfied: kiwisolver>=1.3.1 in
c:\users\surface\anaconda3\lib\site-packages (from matplotlib->lime) (1.4.8)

Requirement already satisfied: pyparsing>=2.3.1 in
c:\users\surface\anaconda3\lib\site-packages (from matplotlib->lime) (3.2.0)

Requirement already satisfied: python-dateutil>=2.7 in
c:\users\surface\anaconda3\lib\site-packages (from matplotlib->lime)
(2.9.0.post0)

Requirement already satisfied: six>=1.5 in c:\users\surface\anaconda3\lib\site-
packages (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)

Requirement already satisfied: colorama in c:\users\surface\anaconda3\lib\site-
packages (from tqdm->lime) (0.4.6)

Requirement already satisfied: shap in c:\users\surface\anaconda3\lib\site-
packages (0.48.0)

Requirement already satisfied: numpy in c:\users\surface\anaconda3\lib\site-
packages (from shap) (2.1.3)

Requirement already satisfied: scipy in c:\users\surface\anaconda3\lib\site-
packages (from shap) (1.15.3)

Requirement already satisfied: scikit-learn in
c:\users\surface\anaconda3\lib\site-packages (from shap) (1.6.1)

Requirement already satisfied: pandas in c:\users\surface\anaconda3\lib\site-
packages (from shap) (2.2.3)

```

Requirement already satisfied: tqdm>=4.27.0 in
c:\users\surface\anaconda3\lib\site-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in
c:\users\surface\anaconda3\lib\site-packages (from shap) (24.2)
Requirement already satisfied: slicer==0.0.8 in
c:\users\surface\anaconda3\lib\site-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in
c:\users\surface\anaconda3\lib\site-packages (from shap) (0.61.0)
Requirement already satisfied: cloudpickle in
c:\users\surface\anaconda3\lib\site-packages (from shap) (3.0.0)
Requirement already satisfied: typing-extensions in
c:\users\surface\anaconda3\lib\site-packages (from shap) (4.12.2)
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in
c:\users\surface\anaconda3\lib\site-packages (from numba>=0.54->shap) (0.44.0)
Requirement already satisfied: colorama in c:\users\surface\anaconda3\lib\site-
packages (from tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in
c:\users\surface\anaconda3\lib\site-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
c:\users\surface\anaconda3\lib\site-packages (from pandas->shap) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in
c:\users\surface\anaconda3\lib\site-packages (from pandas->shap) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\surface\anaconda3\lib\site-
packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
c:\users\surface\anaconda3\lib\site-packages (from scikit-learn->shap) (3.5.0)
Collecting sklearn
  Downloading sklearn-0.0.post12.tar.gz (2.6 kB)
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'error'

error: subprocess-exited-with-error

```

× python setup.py egg_info did not run successfully.

exit code: 1

> [15 lines of output]

The 'sklearn' PyPI package is deprecated, use 'scikit-learn'
rather than 'sklearn' for pip commands.

Here is how to fix this error in the main use cases:

- use 'pip install scikit-learn' rather than 'pip install sklearn'
- replace 'sklearn' by 'scikit-learn' in your pip requirements files (requirements.txt, setup.py, setup.cfg, Pipfile, etc ...)
- if the 'sklearn' package is used by one of your dependencies, it would be great if you take some time to track which package uses 'sklearn' instead of 'scikit-learn' and report it to their issue tracker

- as a last resort, set the environment variable
SKLEARN_ALLOW_DEPRECATED_SKLEARN_PACKAGE_INSTALL=True to avoid this
error

More information is available at
<https://github.com/scikit-learn/sklearn-pypi-package>
[end of output]

note: This error originates from a subprocess, and is likely not a problem
with pip.

error: metadata-generation-failed

× Encountered error while generating package metadata.
> See above for output.

note: This is an issue with the package mentioned above, not pip.

hint: See above for details.

```
[64]: ### Carga y EDA de los datos del CSV ###
df = pd.read_csv("titanic.csv" , delimiter=";")
print("Forma:", df.shape)
print("Columnas:", df.columns.tolist())

# Calidad de datos
print("\nNulos por columna:")
print(df.isna().sum())

# Distribución de la variable objetivo
TARGET = "survived"
assert TARGET in df.columns, "No encuentro la columna objetivo 'survived'."
print("\nDistribución de survived:")
print(df[TARGET].value_counts(dropna=False))
print((df[TARGET].value_counts(normalize=True) * 100).round(1), "%")

# Listas de tipos
num_cols = df.select_dtypes(include=np.number).columns.tolist()
cat_cols = df.select_dtypes(exclude=np.number).columns.tolist()
print("\nNuméricas:", num_cols)
print("Categorías:", cat_cols)

# Estadísticos
print("\nDescripción numérica:")
print(df[num_cols].describe().T)

if cat_cols:
    print("\nDescripción categórica:")
    print(df[cat_cols].describe().T)
```

```
sns.countplot(x="survived", data=df); plt.title("Supervivencia (0/1)"); plt.
    ↪show()
sns.heatmap(df.isna(), cbar=False); plt.title("Mapa de nulos"); plt.show()
```

Forma: (1309, 14)

Columnas: ['pclass', 'survived', 'name', 'sex', 'age', 'sibsp', 'parch', 'ticket', 'fare', 'cabin', 'embarked', 'boat', 'body', 'home.dest']

Nulos por columna:

```
pclass      0
survived    0
name        0
sex         0
age        263
sibsp       0
parch       0
ticket      0
fare        1
cabin     1014
embarked    2
boat       823
body      1188
home.dest   564
dtype: int64
```

Distribución de survived:

```
survived
0      809
1      500
Name: count, dtype: int64
survived
0      61.8
1      38.2
Name: proportion, dtype: float64 %
```

Numéricas: ['pclass', 'survived', 'sibsp', 'parch', 'body']

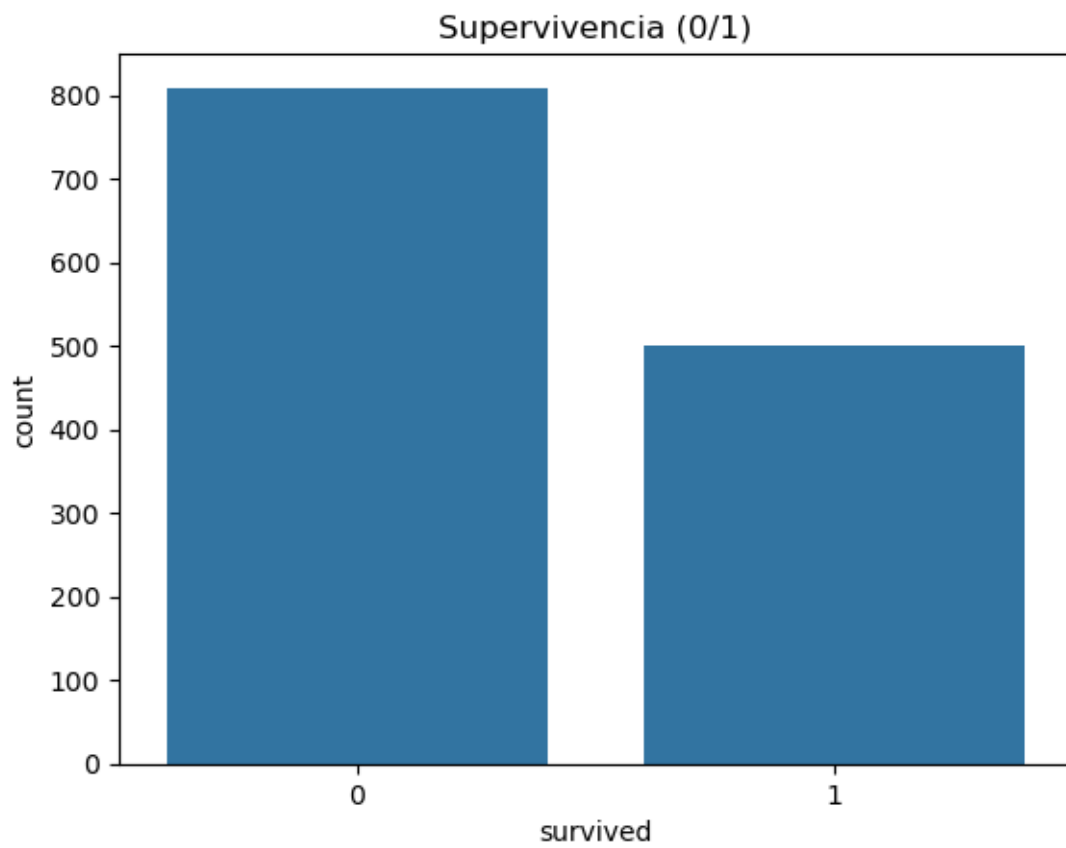
Catóricas: ['name', 'sex', 'age', 'ticket', 'fare', 'cabin', 'embarked', 'boat', 'home.dest']

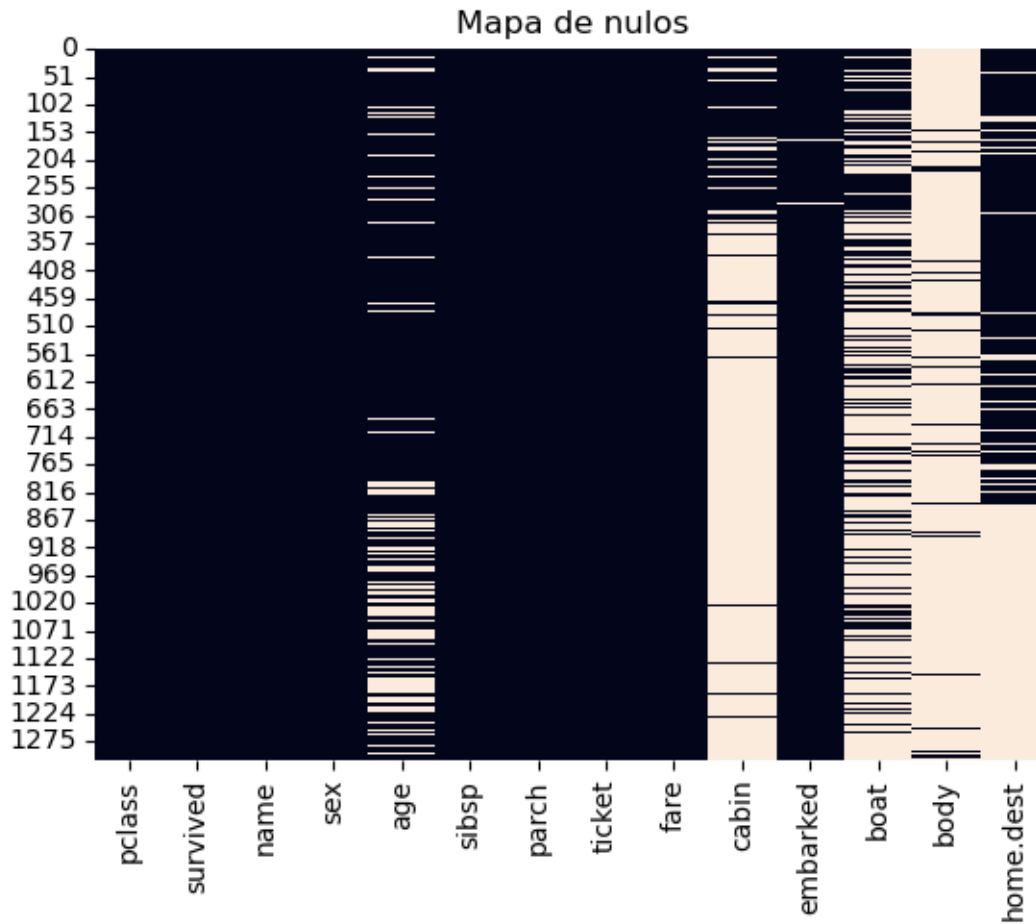
Descripción numérica:

	count	mean	std	min	25%	50%	75%	max
pclass	1309.0	2.294882	0.837836	1.0	2.0	3.0	3.0	3.0
survived	1309.0	0.381971	0.486055	0.0	0.0	0.0	1.0	1.0
sibsp	1309.0	0.498854	1.041658	0.0	0.0	0.0	1.0	8.0
parch	1309.0	0.385027	0.865560	0.0	0.0	0.0	0.0	9.0
body	121.0	160.809917	97.696922	1.0	72.0	155.0	256.0	328.0

Descripción categórica:

	count	unique	top	freq
name	1309	1307	Connolly, Miss. Kate	2
sex	1309	2	male	843
age	1046	98	24	47
ticket	1309	929	CA. 2343	11
fare	1308	281	8,05	60
cabin	295	186	C23 C25 C27	6
embarked	1307	3	S	914
boat	486	27	13	39
home.dest	745	369	New York, NY	64





```
[65]: ### Modelo de Regresión Logística ###
# Seleccionamos solo las variables numéricas
X_num = df.select_dtypes(include=np.number).copy()
assert TARGET in X_num.columns, "La 'survived' debe ser numérica (0/1)."
X = X_num.drop(columns=[TARGET])
y = X_num[TARGET]

# Split estratificado
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=33, stratify=y
)

# Pipeline: imputación (mediana) + escalado + LR
pipe_lr = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    LogisticRegression(max_iter=1000, random_state=42)
```

```

)

pipe_lr.fit(X_train, y_train)

# Evaluación
y_pred = pipe_lr.predict(X_test)
print("\n=== Regresión Logística (numéricas) ===")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=3))

# Curvas ROC / PR (siempre que predict_proba esté disponible)
try:
    RocCurveDisplay.from_estimator(pipe_lr, X_test, y_test)
    plt.title("ROC - LR (numéricas)")
    plt.show()

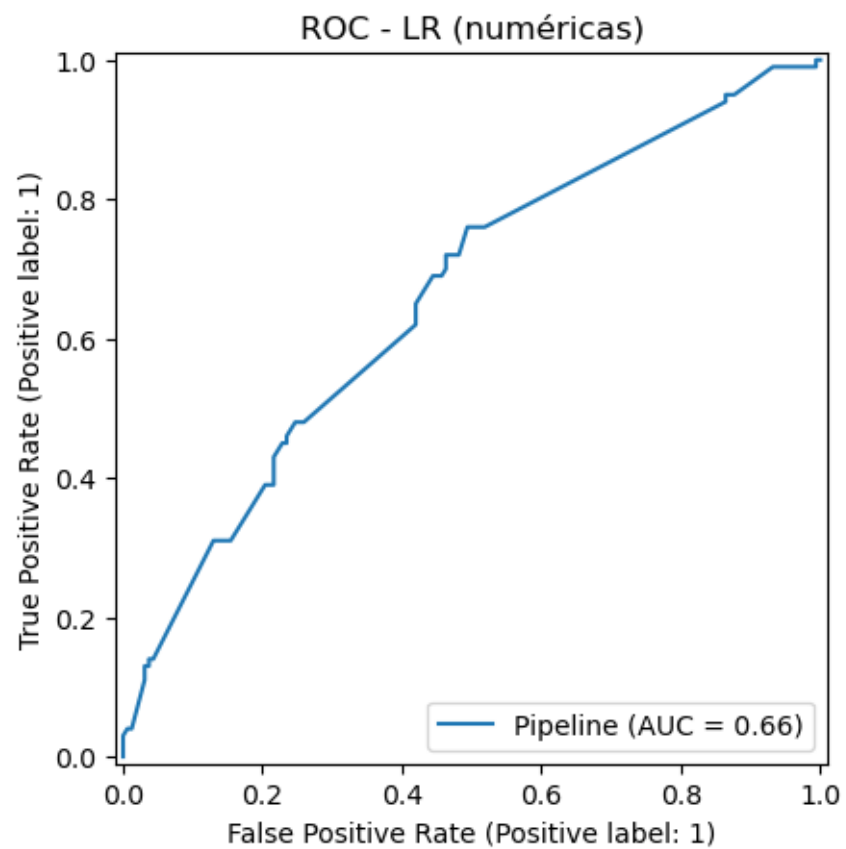
    PrecisionRecallDisplay.from_estimator(pipe_lr, X_test, y_test)
    plt.title("PR - LR (numéricas)")
    plt.show()
except Exception as e:
    print("No se pudieron dibujar curvas ROC/PR:", e)

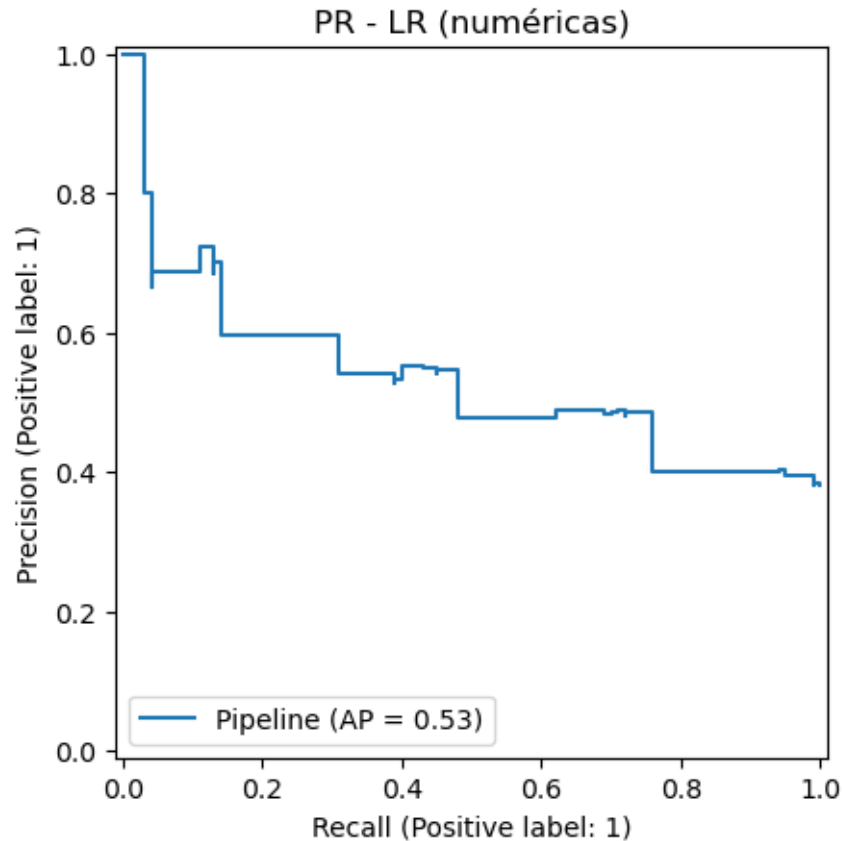
```

=== Regresión Logística (numéricas) ===

Accuracy: 0.648854961832061

	precision	recall	f1-score	support
0	0.694	0.772	0.731	162
1	0.549	0.450	0.495	100
accuracy			0.649	262
macro avg	0.622	0.611	0.613	262
weighted avg	0.639	0.649	0.641	262





```
[93]: ###
#El modelo acierta más cuando predice que alguien NO sobrevivió (recall = 0.77),
#pero falla más al identificar a quienes sí sobrevivieron (recall = 0.45).
#Esto es típico cuando hay clases desbalanceadas (más pasajeros murieron que
↳sobrevivieron)
#El F1-score medio (0.61) refleja un desempeño moderado: mejor que el azar,
↳pero con margen de mejora.
```

```
[85]: ### Interpretabilidad con LIME ###
if lime is not None:
    try:
        # Datos transformados (solo pasos previos al clasificador)
        X_train_tx = pd.DataFrame(
            pipe_lr[:-1].transform(X_train), columns=X.columns, index=X_train.
↳index
        )
        X_test_tx = pd.DataFrame(
            pipe_lr[:-1].transform(X_test), columns=X.columns, index=X_test.
↳index
```

```

)
y_train_tx = y_train.reset_index(drop=True)
y_test_tx = y_test.reset_index(drop=True)

explainer_lr = lime.lime_tabular.LimeTabularExplainer(
    training_data=X_train_tx.values,
    training_labels=y_train_tx.values,
    feature_names=X.columns.tolist(),
    mode="classification"
)

lr_predict = lambda data: pipe_lr[-1].predict_proba(data)

instance_idx = min(1, len(X_test_tx)-1)
instance = X_test_tx.iloc[instance_idx]
instance_true = int(y_test_tx.iloc[instance_idx])
exp = explainer_lr.explain_instance(instance, lr_predict)

print(f"LIME (LR) - instancia {instance_idx}: pred={int(exp.
↪predict_proba[1] > exp.predict_proba[0])}, real={instance_true}")
except Exception as e:
    print("LIME (LR) no se pudo ejecutar:", e)
else:
    print("LIME no disponible (pip install lime).")

```

LIME (LR) - instancia 1: pred=0, real=0

LIME no disponible (pip install lime).

c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:

FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
ret[feature] = int(self.lambdas[feature](ret[feature]))
```

c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:

FutureWarning: Series.__setitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To set a value by position, use `ser.iloc[pos] = value`

```
ret[feature] = int(self.lambdas[feature](ret[feature]))
```

c:\Users\Surface\anaconda3\Lib\site-packages\lime\lime_tabular.py:544:

FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
binary_column = (inverse_column == first_row[column]).astype(int)
```

c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:

FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```

    ret[feature] = int(self.lambdas[feature](ret[feature]))
c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:
FutureWarning: Series.__setitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To set a value by position, use `ser.iloc[pos] = value`
    ret[feature] = int(self.lambdas[feature](ret[feature]))
c:\Users\Surface\anaconda3\Lib\site-packages\lime\lime_tabular.py:427:
FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    discretized_instance[f]])

```

```

[ ]: ### Decision tree ###
df_dt = df.copy()

# convertir números con coma ("1,23") a punto ("1.23") si aparecen en columnas
↪object
for c in df_dt.columns:
    if df_dt[c].dtype == object:
        # si detecta formato numérico con coma en alguna fila, intenta
        ↪convertirlo
        if df_dt[c].astype(str).str.contains(r"^-?\d+,\d+$", na=False).any():
            df_dt[c] = df_dt[c].str.replace(",", ".", regex=False)
            df_dt[c] = pd.to_numeric(df_dt[c], errors="ignore")

# Imputación: numéricas (mediana), categóricas (moda)
for c in df_dt.columns:
    if c == TARGET:
        continue
    if pd.api.types.is_numeric_dtype(df_dt[c]):
        df_dt[c] = df_dt[c].replace([np.inf, -np.inf], np.nan).fillna(df_dt[c].
        ↪median())
    else:
        df_dt[c] = df_dt[c].fillna(df_dt[c].mode().iloc[0])

# Eliminar columnas poco útiles si existen
for col in ["name", "ticket", "cabin"]:
    if col in df_dt.columns:
        df_dt.drop(columns=[col], inplace=True)

# Dummies para categóricas
cat_cols_dt = df_dt.select_dtypes(exclude=np.number).columns.tolist()
df_dt = pd.get_dummies(df_dt, columns=cat_cols_dt, drop_first=True)

# Split y entrenamiento
assert TARGET in df_dt.columns, "No está 'survived' tras el preprocesado."
X2 = df_dt.drop(columns=[TARGET])

```

```

y2 = df_dt[TARGET]

X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.20, random_state=33, stratify=y2
)

dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X2_train, y2_train)

y2_pred = dt.predict(X2_test)
print("\n=== Árbol de Decisión (todas las variables) ===")
print("Accuracy:", accuracy_score(y2_test, y2_pred))
print(classification_report(y2_test, y2_pred, digits=3))

```

```

=== Árbol de Decisión (todas las variables) ===
Accuracy: 0.9389312977099237

```

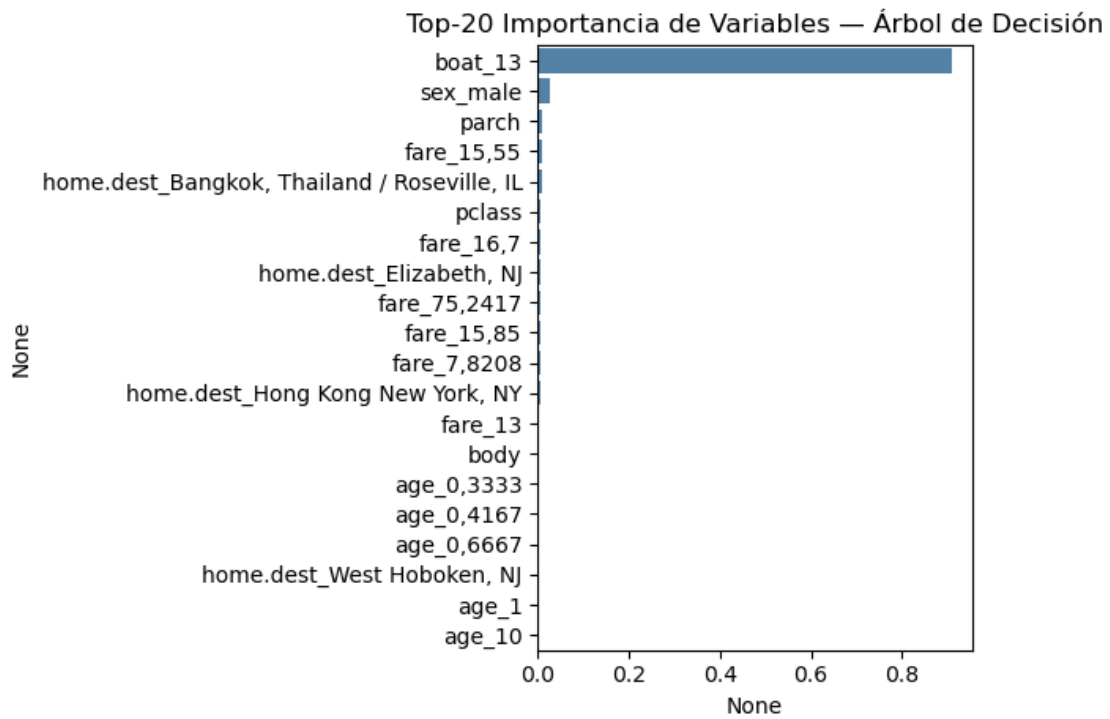
	precision	recall	f1-score	support
0	0.935	0.969	0.952	162
1	0.947	0.890	0.918	100
accuracy			0.939	262
macro avg	0.941	0.930	0.935	262
weighted avg	0.939	0.939	0.939	262

[94]: *# El árbol captura relaciones no lineales entre las variables y la*
↳ supervivencia, por eso mejora mucho respecto a la regresión logística.
Sin embargo, un accuracy del 94 % podría indicar ligero sobreajuste
↳ (overfitting) si los datos de test son pocos o las variables dummificadas
↳ son muchas.

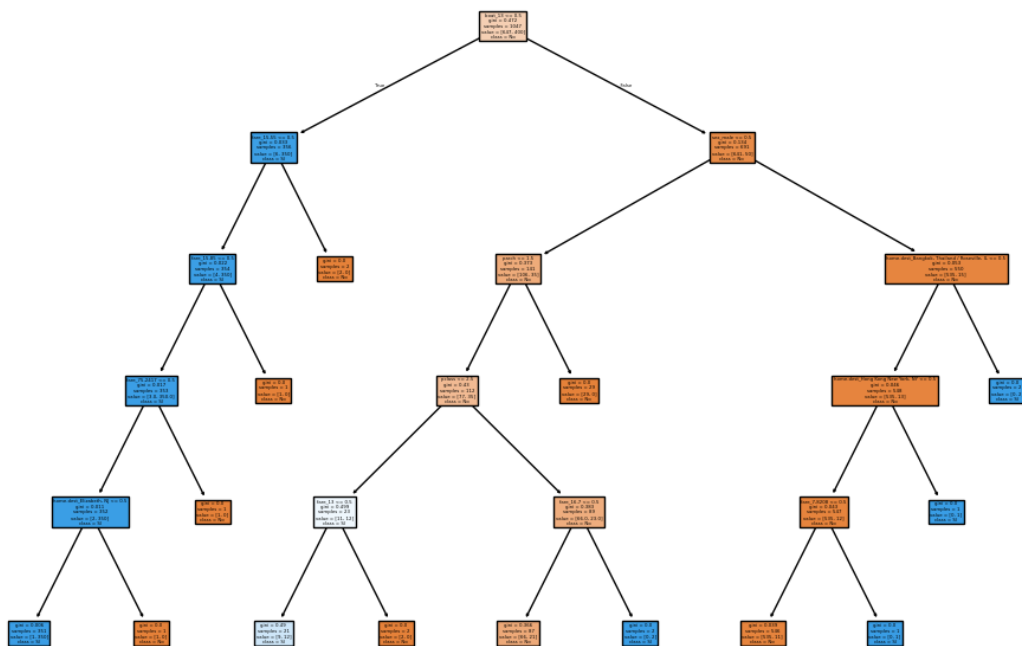
[87]: *### Importancia de variables ###*
Importancias (global)
feature_imp = pd.Series(dt.feature_importances_, index=X2.columns).
↳ sort_values(ascending=False)
sns.barplot(x=feature_imp.head(20), y=feature_imp.index[:20], color="steelblue")
plt.title("Top-20 Importancia de Variables - Árbol de Decisión")
plt.tight_layout()
plt.show()

Visual del tree
fig, ax = plt.subplots(figsize=(12, 8), dpi=100)
tree.plot_tree(dt, feature_names=X2.columns, class_names=["No", "Sí"],
↳ filled=True)
plt.title("Árbol de Decisión (max_depth=5)")

```
plt.show()
```



Árbol de Decisión (max_depth=5)



```
[95]: ### En este caso podemos ver como las variables más importantes se van  
↪ desglosando en ramas hasta las menos importantes.
```

```
[92]: ### Tree con LIME y SHAP ###  
# LIME (árbol)  
if lime is not None:  
    try:  
        explainer_dt = lime.lime_tabular.LimeTabularExplainer(  
            training_data=X2_train.values,  
            training_labels=y2_train.values,  
            feature_names=X2.columns.tolist(),  
            mode="classification"  
        )  
        dt_predict = lambda data: dt.predict_proba(data)  
  
        idx = min(2, len(X2_test)-1)  
        instance = X2_test.iloc[idx]  
        instance_true = int(y2_test.iloc[idx])  
  
        exp_dt = explainer_dt.explain_instance(instance, dt_predict)  
        # En Jupyter: exp_dt.show_in_notebook(show_all=False)  
        print(f"LIME (DT) - instancia {idx}: pred={int(exp_dt.predict_proba[1])  
↪ exp_dt.predict_proba[0])}, real={instance_true}")  
    except Exception as e:  
        print("LIME (DT) no se pudo ejecutar:", e)  
else:  
    print("LIME no disponible (pip install lime).")  
  
# SHAP (árbol)  
if shap is not None:  
    try:  
        explainer_shap = shap.TreeExplainer(dt)  
        shap_values = explainer_shap.shap_values(X2_test)  
  
        # Local  
        try:  
            shap.initjs()  
        except Exception:  
            pass  
        idx = min(33, len(X2_test)-1)  
        shap.force_plot(explainer_shap.expected_value[1], shap_values[1][idx],  
↪ X2_test.iloc[[idx]])  
  
        # Global
```

```

        shap.summary_plot(shap_values, X2_test)
    except Exception as e:
        print(" SHAP no se pudo ejecutar:", e)
else:
    print("SHAP no disponible (pip install shap).")

```

```

c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:
FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    ret[feature] = int(self.lambdas[feature](ret[feature]))
c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:
FutureWarning: Series.__setitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To set a value by position, use `ser.iloc[pos] = value`
    ret[feature] = int(self.lambdas[feature](ret[feature]))
c:\Users\Surface\anaconda3\Lib\site-packages\lime\lime_tabular.py:544:
FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    binary_column = (inverse_column == first_row[column]).astype(int)
c:\Users\Surface\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739:
UserWarning: X does not have valid feature names, but DecisionTreeClassifier was
fitted with feature names
    warnings.warn(
c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:
FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    ret[feature] = int(self.lambdas[feature](ret[feature]))
c:\Users\Surface\anaconda3\Lib\site-packages\lime\discretize.py:110:
FutureWarning: Series.__setitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To set a value by position, use `ser.iloc[pos] = value`
    ret[feature] = int(self.lambdas[feature](ret[feature]))
c:\Users\Surface\anaconda3\Lib\site-packages\lime\lime_tabular.py:427:
FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a
future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    discretized_instance[f]])

LIME (DT) - instancia 2: pred=1, real=1

<IPython.core.display.HTML object>

SHAP no se pudo ejecutar: Length of features is not equal to the length of
shap_values!

```


[]: